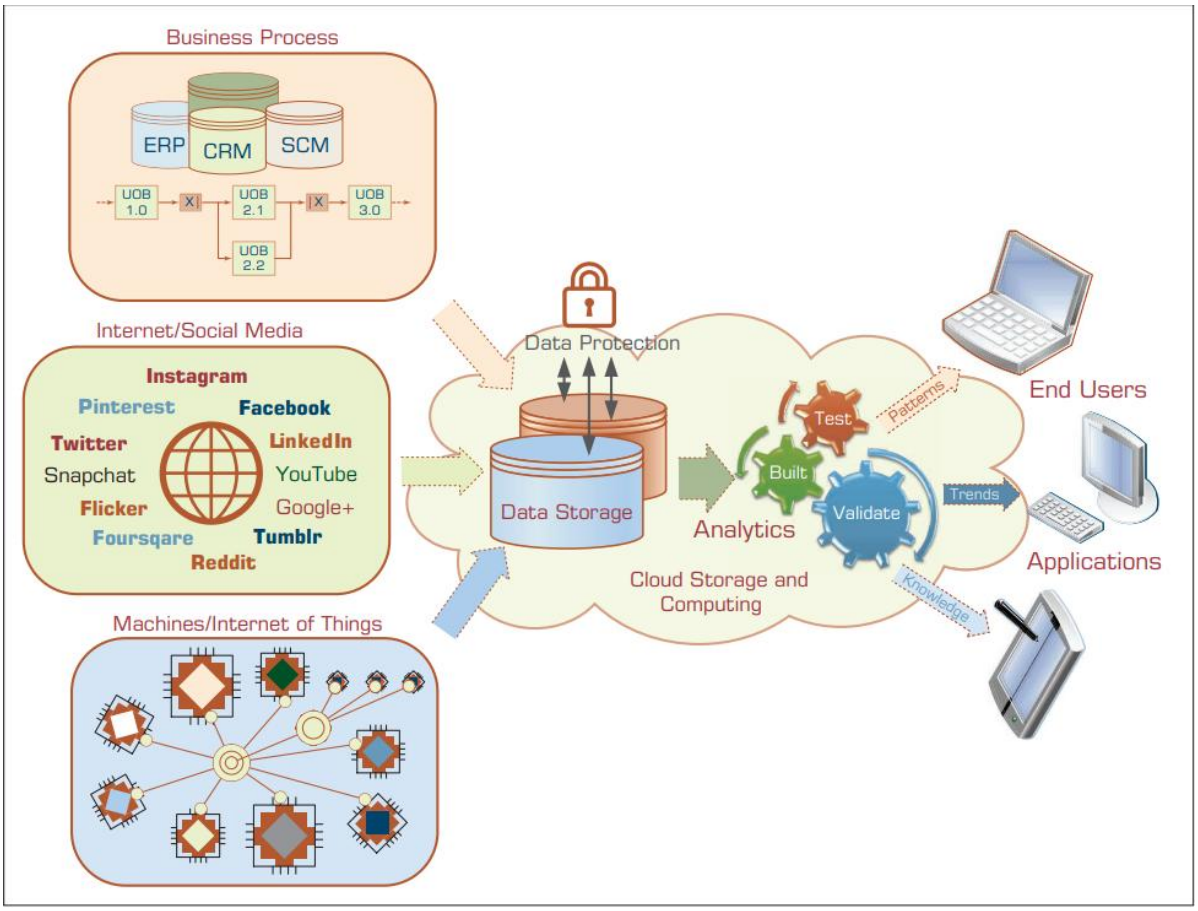
Predictive Analysis

Study Notes | Chapter 2

1. The Nature of Data

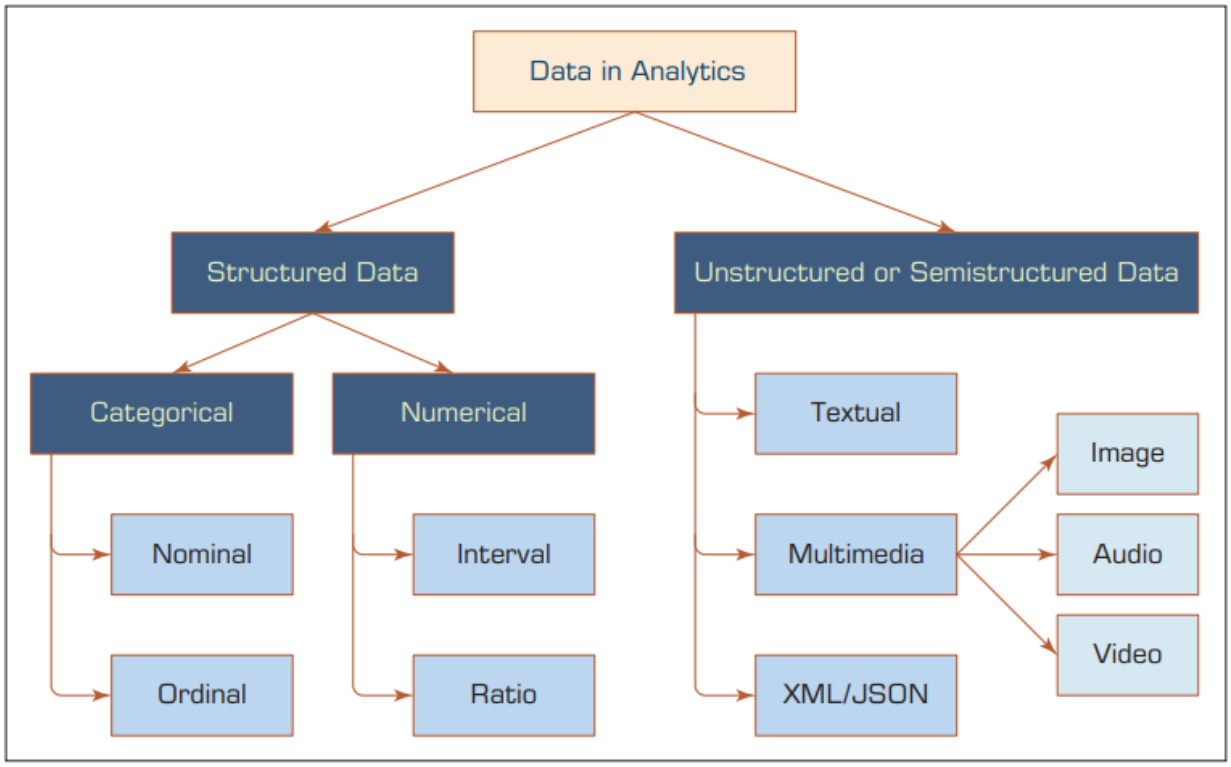
Data is the starting point for any Business Intelligence (BI), data science, or analytics work. Think of data as raw material. When you process and organize it, it becomes information. When you use that information to understand something, it becomes knowledge.

Data comes from many places: business systems (ERP, CRM, SCM), social media platforms like Facebook and Twitter, machines connected to the internet (IoT), and cloud storage systems.



2. Types of Data

At a basic level, data is divided into two main types: Structured and Unstructured.



Unstructured Data

This is data that does not have a fixed format. It can be text, images, audio, video, or web content. It is harder for computers to process directly.

Structured Data

This is organized data that fits neatly into rows and columns, like a spreadsheet or database. Data mining tools work with structured data. Structured data is further divided into two groups:

Categorical Data

Categorical data puts things into groups or labels. There are two kinds:

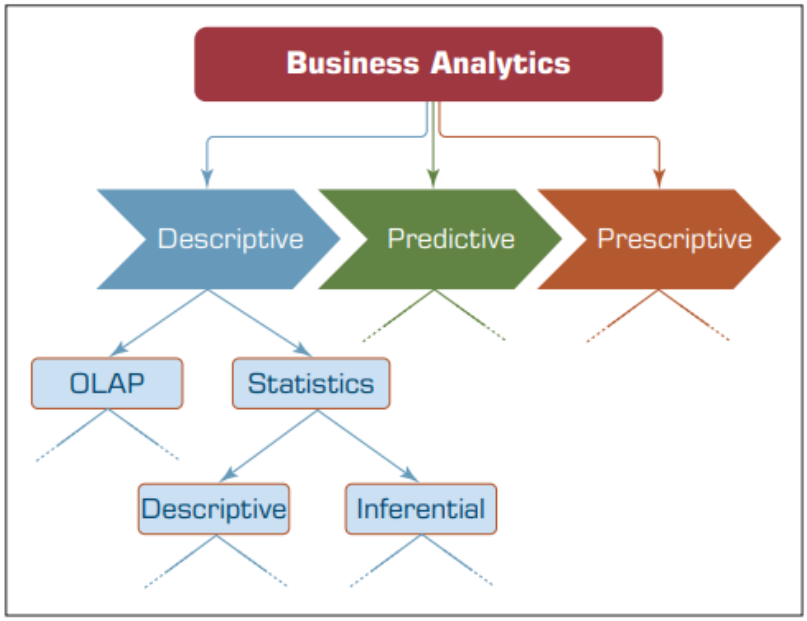
- Nominal data: labels with no order. Example: marital status (single, married, divorced), or colors (brown, green, blue).
- Ordinal data: labels that have a rank or order. Example: satisfaction level (low, medium, high), or education level (high school, college, graduate).

Numeric Data

Numeric data uses numbers to measure something. It can be:

- Integer: whole numbers only. Example: number of children.
- Real (continuous): includes decimal values. Example: temperature, distance.

- Interval data: measured on a scale where the difference between values is meaningful, but there is no true zero. Example: temperature in Celsius.
- Ratio data: like interval but with a true zero point. Example: weight, height, time.

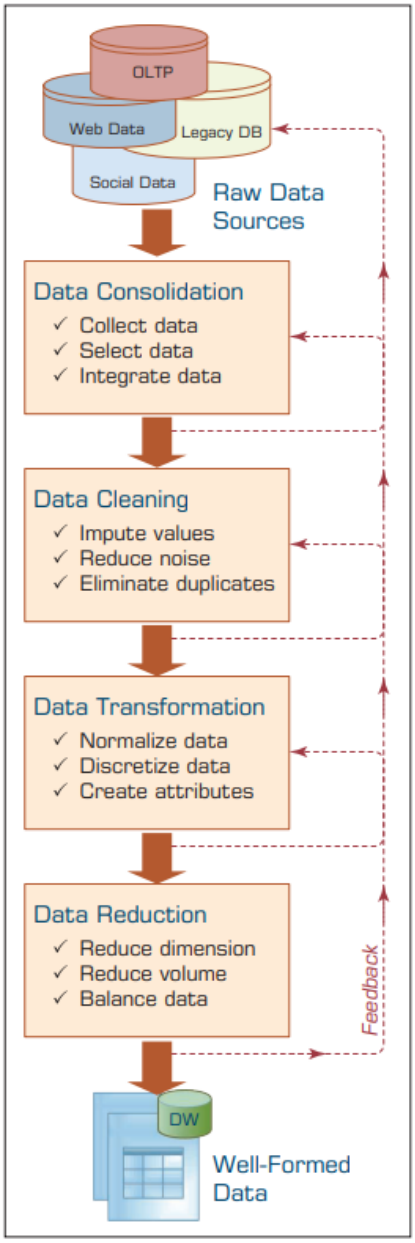


3. Data Preprocessing

Before data can be used for analysis, it usually needs to be cleaned and prepared. This is called data preprocessing. Raw data is often messy, incomplete, or inconsistent. The goal of preprocessing is to turn raw data into well-formed, usable data.

TABLE 2.1 A Summary of Data Preprocessing Tasks and Potential Methods

Main Task	Subtasks	Popular Methods
Data consolidation	Access and collect the data	SQL queries, software agents, Web services.
	Select and filter the data	Domain expertise, SQL queries, statistical tests.
	Integrate and unify the data	SQL queries, domain expertise, ontology-driven data mapping.
Data cleaning	Handle missing values in the data	Fill in missing values (imputations) with most appropriate values (mean, median, min/max, mode, etc.); recode the missing values with a constant such as "ML"; remove the record of the missing value; do nothing.
	Identify and reduce noise in the data	Identify the outliers in data with simple statistical techniques (such as averages and standard deviations) or with cluster analysis; once identified, either remove the outliers or smooth them by using binning, regression, or simple averages.
	Find and eliminate erroneous data	Identify the erroneous values in data (other than outliers), such as odd values, inconsistent class labels, odd distributions; once identified, use domain expertise to correct the values or remove the records holding the erroneous values.
Data transformation	Normalize the data	Reduce the range of values in each numerically valued variable to a standard range (e.g., 0 to 1 or −1 to +1) by using a variety of normalization or scaling techniques.
	Discretize or aggregate the data	If needed, convert the numeric variables into discrete representations using range- or frequency-based binning techniques; for categorical variables, reduce the number of values by applying proper concept hierarchies.
	Construct new attributes	Derive new and more informative variables from the existing ones using a wide range of mathematical functions (as simple as addition and multiplication or as complex as a hybrid combination of log transformations).
Data reduction	Reduce number of attributes	Principal component analysis, independent component analysis, chi-square testing, correlation analysis, and decision tree induction.
	Reduce number of records	Random sampling, stratified sampling, expert-knowledge-driven purposeful sampling.
	Balance skewed data	Oversample the less represented or undersample the more represented classes.



- Data Consolidation: Collect data from different sources, select what is needed, and combine it all together using SQL queries, software agents, or web services.
- Data Cleaning: Handle missing values (fill them in or remove them), reduce noise, and remove duplicate records.
- Data Transformation: Normalize values so they fall in a standard range (0 to 1 or -1 to +1), and create new useful attributes from existing ones.
- Data Reduction: Reduce the size of the data while keeping the important parts, so analysis is faster and easier.

4. Regression Modeling

Regression is one of the most commonly used tools in data analytics. It helps us understand and measure the relationship between variables. In simple terms: regression tries to find how one thing (the output) changes when another thing (the input) changes.

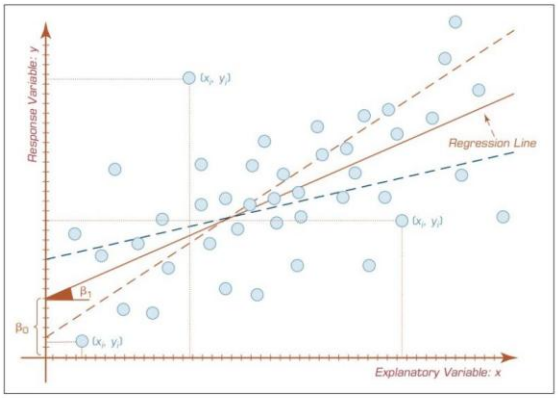
- Hypothesis testing: To check whether two or more variables are related to each other.
- Prediction or forecasting: To estimate future values based on current data.

Simple vs Multiple Regression

- Simple Regression: One input variable predicts one output variable. Example: height predicts weight.
- Multiple Regression: More than one input variable predicts the output. Example: height, BMI, and gender together predict weight.

How the Regression Line is Found

The most common method to find the best-fit line is called Ordinary Least Squares (OLS). It finds the line that minimizes the total error between the actual data points and the predicted values.



Even though there are several methods/algorithms proposed to identify the regression line, the one that is most commonly used is called the **ordinary least squares (OLS) method**.

How Do We Know If the Model is Good?

- R-squared (R2): Tells you how much of the variation in the output is explained by the model. A value of 0 means no fit; a value of 1 means a perfect fit.
- Overall F-test: Checks whether the model as a whole is statistically useful.
- Root Mean Square Error (RMSE): Measures the average size of the errors in prediction. Lower is better.

Key Assumptions in Linear Regression

- Linearity: The relationship between input and output is a straight line.
- Independence: Each data point is independent of others.
- Normality: The errors (residuals) follow a normal distribution.
- Constant Variance: The spread of errors stays the same across all values.
- No Multicollinearity: Input variables should not be too closely related to each other.

5. Logistic Regression

Logistic Regression (LR) is used when the output is a category, not a number. Instead of predicting a value, it predicts which group something belongs to. For example: Will a customer buy a product? (Yes or No). Will a patient get a disease? (Positive or Negative).

- The output is a class or category, not a number.
- The original form handles binary output (two choices: yes/no, 0/1, pass/fail).
- A modern version called Multinomial LR can handle more than two categories.
- It is widely used in medicine, social science, and business analytics.

6. Business Reporting

KEY DEFINITION

> Definition: Business reporting (also referred to as OLAP — Online Analytical Processing — or Business Intelligence / BI) is an essential component of the larger drive toward improved, evidence-based, and optimal managerial decision making. It involves transforming raw organizational data into actionable reports and summaries for decision makers.

6.1 Purpose and Importance

Decision makers require accurate, timely, and contextualized information to make well-informed business decisions. Information is essentially the contextualization of data: raw facts become meaningful when organized and interpreted. In addition to statistical methods, information (descriptive analytics) can also be obtained using Online Transaction Processing (OLTP) systems.

Business reporting provides a structured pipeline through which organizational data is transformed into visual reports, dashboards, and summaries that support managerial decisions at all levels of an organization.

6.2 Data Sources

The foundation of business reports is data drawn from both internal and external sources:

- **Internal sources:** Online Transaction Processing (OLTP) systems such as ERP, CRM, and SCM platforms that record day-to-day operations.
- **External sources:** Market data, social media feeds, government publications, and third-party databases.

6.3 The ETL Process

Creating business reports involves the ETL process, which prepares and loads data into a centralized data warehouse before reporting tools generate the final outputs.

Step	Full Name	Description
Extract (E)	Data Extraction	Pull data from various OLTP systems and other internal/external sources.
Transform (T)	Data Transformation	Clean, format, normalize, and restructure the data for consistency and usability.
Load (L)	Data Loading	Load the processed data into a central data warehouse repository.
Report	Report Generation	Use BI/reporting tools to generate dashboards, charts, and analytical summaries.

6.4 Business Reporting Flow

The complete flow from raw transactional data to managerial decisions follows a well-defined pipeline:

- Business functions generate transactional records and exception events.
- These records are stored in data repositories (data warehouses).
- BI reporting tools process the warehouse data into visual reports and dashboards.
- Decision makers consume these reports to determine appropriate actions.

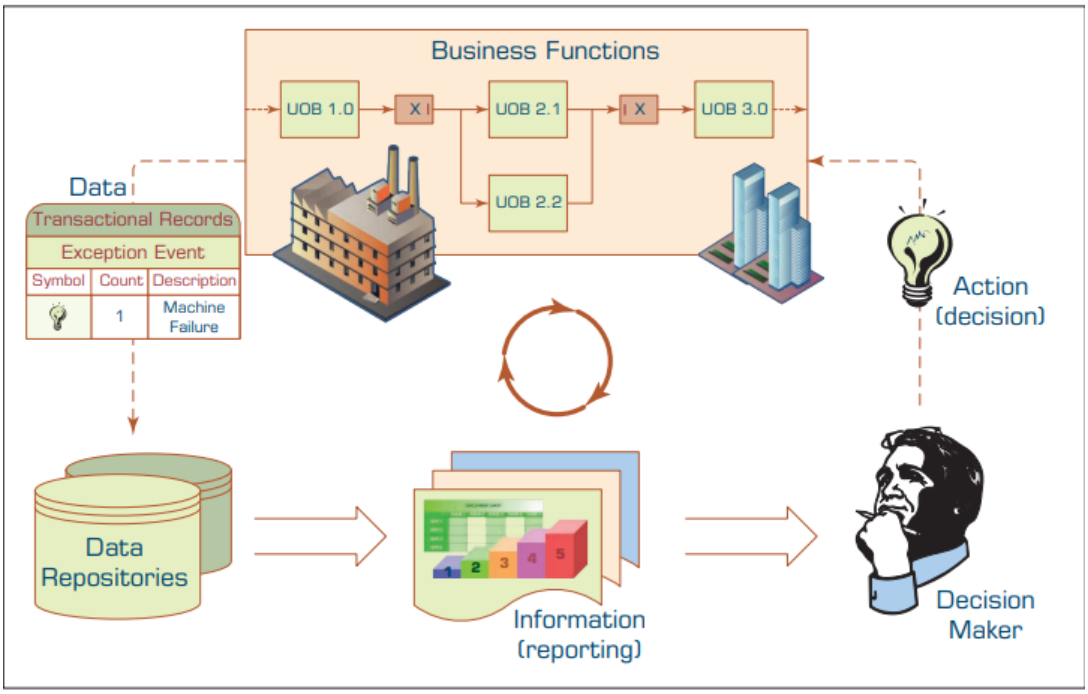


Figure 2: Slide 17 — Business Reporting Process Flow Diagram: From Transactional Data to Decision Making (Source: Lecture Slide, Page 17)

7. Data Visualization and Dashboards

7.1 Data Visualization

KEY DEFINITION

> Definition: Data visualization (also called information visualization) is defined as "the use of visual representations to explore, make sense of, and communicate data." Although the term "data visualization" is commonly used, what is typically depicted in visualizations is information — not raw data — since information represents the aggregation, summarization, and contextualization of data (raw facts).

The primary goal of data visualization is to leverage the human visual system to enable rapid perception of patterns, trends, correlations, and anomalies within large and complex data sets. Effective visualizations translate abstract numerical data into intuitive graphical forms that are immediately comprehensible to decision makers.

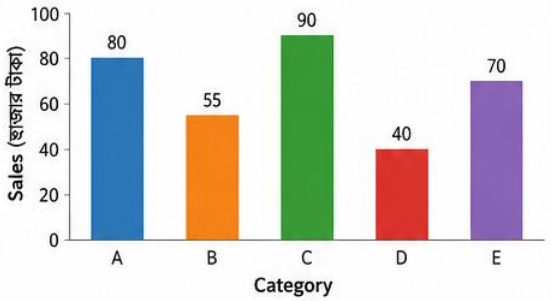
7.2 Different Types of Charts and Graphs

Charts and graphs are the primary tools of data visualization. Different chart types serve different analytical purposes. The selection of an appropriate chart type depends on the nature of the data and the analytical question being addressed.

Chart / Graph Type	Best Used For	Example Use Case
Bar Chart	Comparing discrete categories	Sales by product category
Line Chart	Showing trends over time	Monthly revenue over a year
Pie / Donut Chart	Showing proportions of a whole	Market share by company
Scatter Plot	Showing correlation between two variables	Height vs. Weight
Histogram	Showing frequency distribution	Distribution of exam scores
Heat Map	Showing density or intensity across two dimensions	Website click patterns
Box Plot	Showing statistical distribution and outliers	Salary range by department
Bubble Chart	Three-variable comparison (x, y, and size)	GDP, life expectancy, population

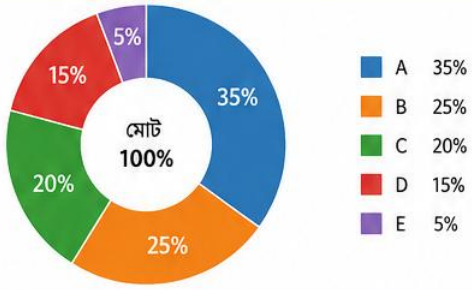
Treemap	Hierarchical data as nested rectangles	Budget breakdown by department
Gantt Chart	Project timelines and scheduling	Project management tasks

1. Bar Chart → ক্যাটাগরি তুলনা

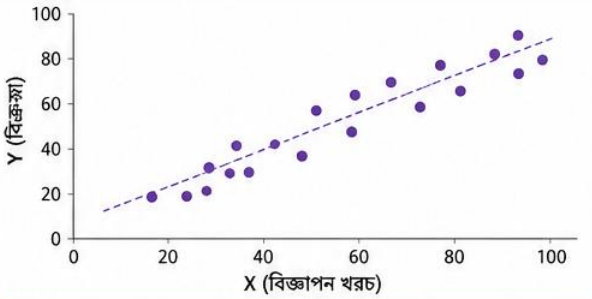


2. Line Chart → সময় অনুযায়ী ট্রেন্ড

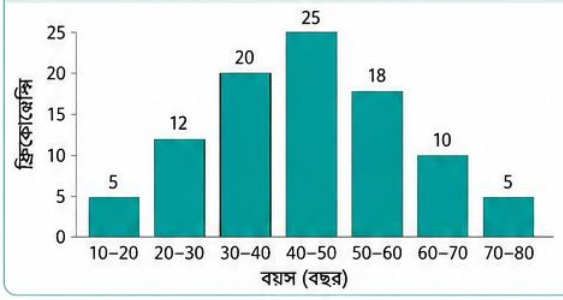
3. Pie/Donut Chart → মোটের অংশ দেখানো



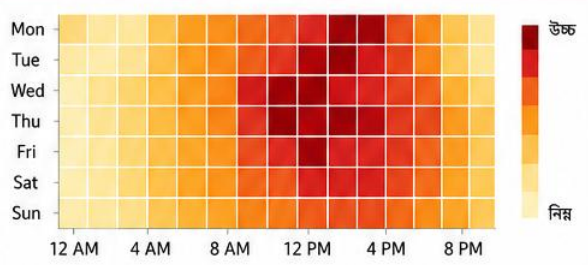
4. Scatter Plot → দুই ভেরিয়েবলের সম্পর্ক



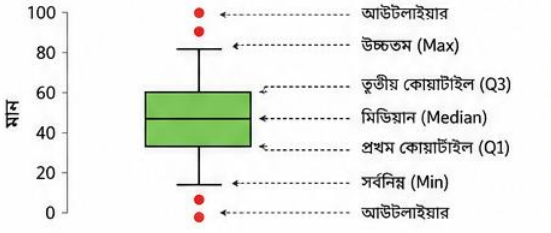
5. Histogram → ফ্রিকোয়েন্সি ডিস্ট্রিবিউশন



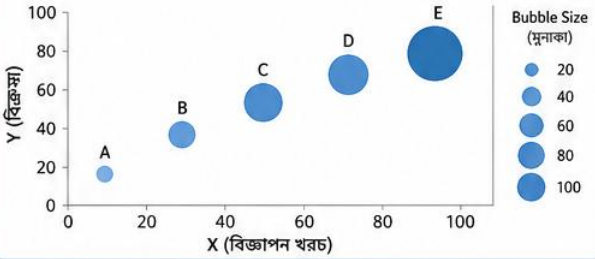
6. Heat Map → ঘনত্ব বা ইন্টেনসিটি



7. Box Plot → স্ট্যাটিস্টিকাল ডিস্ট্রিবিউশন ও আউটলাইয়ার



8. Bubble Chart → তিন ভেরিয়েবল তুলনা



9. Treemap → হায়াররকিক্যাল ডেটা

10. Gantt Chart → প্রজেক্ট টাইমলাইন

7.3 The Emergence of Visual Analytics

KEY DEFINITION

> Visual Analytics: A discipline that combines interactive data visualization with analytical techniques, enabling users to explore, analyze, and communicate insights from large, multi-dimensional data sets stored in data warehouses. Visual analytics goes beyond traditional charts by supporting interactive exploration across multiple dimensions and millions of records.

As noted by Seth Grimes (2009), there is a growing palette of data visualization techniques and tools that enable users of business analytics and BI systems to better communicate relationships, add historical context, uncover hidden correlations, and tell persuasive stories that clarify and call to action.

7.3.1 Key Challenges in Visualization

In BI and analytics, the fundamental challenges for visualization have revolved around the intuitive representation of large, complex data sets with multiple dimensions and measures:

- **Dimensionality limitation:** Typical charts and graphs involve only two dimensions (x and y axis), sometimes three. This is insufficient for the complexity of modern business data.
- **Data volume gap:** Data in BI systems resides in data warehouses involving a range of dimensions (e.g., product, location, organizational structure, time), a range of measures, and millions of cells of data.
- **Interactivity gap:** Static visualizations do not allow users to drill down, filter, or explore data interactively.
- **Research response:** A number of researchers have developed a variety of new visualization techniques and tools to address these challenges, giving rise to the field of visual analytics.

Feature	Traditional Visualization	Visual Analytics
Dimensions handled	2–3 dimensions	Many dimensions (multi-dimensional)
Data volume	Small to medium data subsets	Millions of records from data warehouses
Interactivity	Static charts	Interactive drill-down and filtering
Analysis type	Descriptive (what happened)	Exploratory and explanatory
User role	Passive viewer	Active analyst

7.4 Information Dashboards

KEY DEFINITION

> Definition: Information dashboards are visual displays of important information that is consolidated and arranged on a single screen so that information can be digested at a single glance and easily explored further. They are common components of most BI and

business analytics platforms, business performance management systems, and performance measurement software suites.

According to Eckerson (2006), a well-designed dashboard is not merely a collection of charts; it is a structured, layered information system organized to serve distinct management needs. A well-designed dashboard has three distinct layers:

Layer	Name	Data Type	Purpose
Layer 1	Monitoring	Graphical, abstracted data	Track key performance metrics (KPIs) at a glance using gauges, traffic lights, and sparklines.
Layer 2	Analysis	Summarized dimensional data	Analyze the root cause of problems by drilling into summarized data.
Layer 3	Management	Detailed operational data	Identify specific actions to take in order to resolve problems.

7.4.1 Dashboard Design Principles

Effective information dashboards follow established design principles to ensure clarity, usability, and actionability:

- **Single-screen principle:** All key information must be visible on a single screen without scrolling, enabling a true "at-a-glance" experience.
- **Drill-down capability:** Users should be able to click on summary data to explore underlying details.
- **Appropriate visual encoding:** Choose chart types that match the nature of the data (e.g., gauges for KPIs, line charts for trends, bar charts for comparisons).
- **Minimal visual clutter:** Avoid decorative elements that do not convey information. Every visual element should serve a purpose.
- **Real-time or near-real-time data:** Dashboards should reflect current or recent data to support timely decision making.
- **Role-based views:** Different levels of management (operational, tactical, strategic) may require different dashboard layouts and levels of detail.

Quick Revision Summary — Newly Added Topics

Business Reporting (Section 6)

- **Definition:** Turning organizational data into structured reports for decision makers; also called OLAP or BI.

- **ETL Process:** Extract (pull data) → Transform (clean and format) → Load (into data warehouse) → Report (via BI tools).
- **Data sources:** Internal (OLTP systems: ERP, CRM, SCM) and external sources.
- **Goal:** Evidence-based, optimal managerial decision making.

Data Visualization (Section 7.1)

- **Definition:** Use of visual representations to explore, make sense of, and communicate data (technically: information visualization).
- **Key distinction:** Visualizations show information (aggregated/summarized data), not raw data.
- **Purpose:** Make patterns, trends, and anomalies immediately visible to decision makers.

Types of Charts and Graphs (Section 7.2)

- Bar chart → compare categories; Line chart → show trends over time.
- Scatter plot → show correlation; Histogram → show frequency distribution.
- Pie chart → show proportions; Heat map → show density/intensity.
- Box plot → statistical distribution; Bubble chart → three-variable comparison.

The Emergence of Visual Analytics (Section 7.3)

- **Visual analytics =** data visualization + analytical tools + interactivity.
- **Limitation of traditional charts:** Only 2–3 dimensions; small data subsets.
- **Visual analytics advantage:** Handles millions of records, multiple dimensions, interactive drill-down.
- **Key researcher:** Seth Grimes (2009) — coined the "growing palate" of visualization techniques.

Information Dashboards (Section 7.4)

- **Definition:** A single-screen visual display consolidating key information for at-a-glance decision support.
- **Layer 1 — Monitoring:** Track KPIs graphically.
- **Layer 2 — Analysis:** Identify root causes using summarized dimensional data.
- **Layer 3 — Management:** Determine specific actions from detailed operational data.
- **Design principles:** Single-screen; drill-down capable; appropriate chart types; minimal clutter; near-real-time data.

8. NumPy

What is NumPy?

NumPy stands for Numerical Python. It is a Python library used to work with arrays (lists of numbers). It is the foundation of almost all data science work in Python. You import it using: import numpy as np (or any short name you choose).

```
[3]: print("RASHED")

     RASHED

[5]: import numpy as RNA
     rashed_array = RNA.array([10, 20, 30, 40, 50])
     print(rashed_array)

     [10 20 30 40 50]

[ ]:
```

Numpy library:

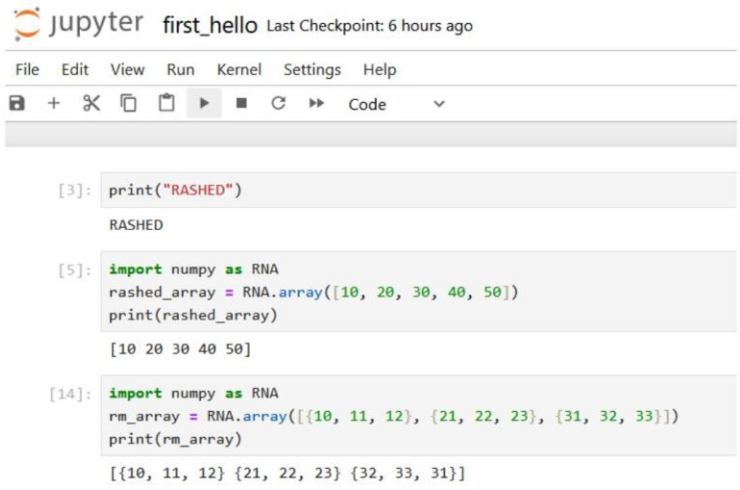
→NumPy is a Python library
→NumPy is used for working with arrays
→NumPy is short for "Numerical Python"
https://www.w3schools.com/python/numpy/default.asp

Creating Arrays

An array is a list of numbers stored in order. NumPy lets you create 1D, 2D, and 3D arrays.

- 1D array (one row of numbers): RNA.array([10, 20, 30, 40, 50])
- 2D array (rows and columns, like a table): RNA.array([[10,11,12],[21,22,23],[31,32,33]])
- 3D array (multiple tables stacked): RNA.array([[[10,11,12],[21,22,23]],[[31,32,33],[41,42,43]]])

Note: Python variable names cannot start with a number. For example, "2_RNA" causes a SyntaxError.



Finding the Shape of an Array

The shape of an array tells you how many rows and columns it has. Use .shape to find it.

Example: A 3D array with shape (2, 2, 3) means 2 blocks, each with 2 rows and 3 columns.

Find out the shape of the matrix

```
[23]: import numpy as RNA
three_d_array = RNA.array([[[10, 11, 12],[21, 22, 23]],[[31, 32, 33], [41, 42, 43]]])
print(three_d_array.shape)
```

```
(2, 2, 3)
```

Indexing and Slicing Arrays

You can access any element in an array using its position number (index). Counting starts from 0.

- array[1] gives the second element. Example: rashed_array[1] from [100,200,300,400,500] gives 200.
- array[1:4] gives elements from position 1 up to (not including) position 4. Example: [200, 300, 400].

```
[2]: import numpy as RNA
rashed_array = RNA.array([100, 200, 300, 400, 500])
f_e = rashed_array[1]
print(f_e)
```

```
200
```

```
[4]: import numpy as RNA
rashed_array = RNA.array([100, 200, 300, 400, 500])
f_e = rashed_array[1:4]
print(f_e)
```

```
[200 300 400]
```

Special Arrays

- Zeros matrix: RNA.zeros(20).reshape(5,4) creates a matrix of 20 zeros arranged in 5 rows and 4 columns.
- Ones matrix: RNA.ones((4,4)) creates a 4x4 matrix filled with ones. You can also change specific positions afterward (broadcast).
- Full matrix: RNA.full((5,6), 99) creates a 5x6 matrix where every cell has the value 99.
- Identity matrix: RNA.identity(5) creates a 5x5 matrix with 1s on the diagonal and 0s everywhere else.

```
[11]: import numpy as RNA
      RNA.zeros(20).reshape(5,4)
```

Problem-1

To create zero matrix

```
[11]: array([[0., 0., 0., 0.],
             [0., 0., 0., 0.],
             [0., 0., 0., 0.],
             [0., 0., 0., 0.],
             [0., 0., 0., 0.]])
```

```
[13]: import numpy as RNA
      RNA.full((5,6), 99)
```

Problem-2

```
[13]: array([[99, 99, 99, 99, 99, 99],
             [99, 99, 99, 99, 99, 99],
             [99, 99, 99, 99, 99, 99],
             [99, 99, 99, 99, 99, 99],
             [99, 99, 99, 99, 99, 99]])
```

To create identity matrix

```
[15]: import numpy as RNA
      RNA.identity(5)
```

```
[15]: array([[1., 0., 0., 0., 0.],
             [0., 1., 0., 0., 0.],
             [0., 0., 1., 0., 0.],
             [0., 0., 0., 1., 0.],
             [0., 0., 0., 0., 1.]])
```

Reference. https://docs.python.org/3/library/random.html

```
[21]: import numpy as RNA
      p_RNA = RNA.random.rand(4,2)
      print(p_RNA)
```

```
[[0.06405883 0.45252315]
 [0.11030007 0.97701463]
 [0.33941876 0.33858928]
 [0.71671714 0.84240548]]
```

Random Arrays

- RNA.random.rand(4,2): Creates a 4x2 matrix filled with random decimal numbers between 0 and 1.
- RNA.random.randint(10, 100, 12): Creates an array of 12 random whole numbers between 10 and 99.
- RNA.arange(12): Creates an array from 0 to 11.
- RNA.arange(2, 12, 2): Creates an array starting at 2, ending before 12, stepping by 2. Result: [2, 4, 6, 8, 10].

Reference. https://docs.python.org/3/library/random.html

```
[29]: import numpy as RNA
      p_RNA = RNA.random.randint(10, 100, 12)
      print(p_RNA)
```

```
[56 16 82 45 75 52 89 16 68 47 17 34]
```

Reference. https://www.geeksforgeeks.org/numpy-arrange-in-python/

```
[2]: import numpy as RNA
     a_RNA = RNA.arange(12)
     print(a_RNA)

     b_RNA = RNA.arange(2, 12, 2)
     print(b_RNA)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[ 2  4  6  8 10]
```

Math Operations on Arrays

- RNA.sum(array): Adds up all values. Example: sum of [20,30,10,40] = 100.
- RNA.mean(array): Finds the average. Example: mean of [20,30,10,40] = 25.0.
- RNA.max(array): Finds the largest value. Example: max of [20,30,10,40] = 40.
- RNA.min(array): Finds the smallest value.
- RNA.sqrt(array): Finds the square root of each element. Example: sqrt of [25,9,16,100] = [5,3,4,10].
- RNA.std(array): Finds the standard deviation (how spread out the values are).
- RNA.sin(), RNA.cos(), RNA.tan(): Trigonometric functions applied to each element.

```
import numpy as RNA
rashed_array = RNA.array([20, 30, 10, 40])
result = RNA.sum(rashed_array)
print(result)
```

to find the sum of all values of declared array

```
100
```

```
[6]: import numpy as RNA
     rashed_array = RNA.array([20, 30, 10, 40])
     result = RNA.mean(rashed_array)
     print(result)

     25.0
```

to find out average

```
[8]: import numpy as RNA
     rashed_array = RNA.array([20, 30, 10, 40])
     result = RNA.max(rashed_array)
     print(result)

     40
```

to find the max. value from the given array

- Do yourself to find the min. value from the given array
- Find the array-index of the smallest number

```
[10]: import numpy as RNA
      rashed_array = RNA.array([25, 9, 16, 100])
      result = RNA.sqrt(rashed_array)
      print(result)

      [ 5.  3.  4. 10.]
```

to find out root square

- Find out sin, cosine, tan value

```
[17]: import numpy as RNA
      rashed_array = RNA.array([25, 9, 16, 100, 4, 36, 49])
      result = RNA.std(rashed_array)
      print(result)

      30.507275813337063
```

? ? ?

Broadcast Array and Array Arithmetic

You can change more than one index value at a time in an array. This is called broadcasting.

You can also do arithmetic between two arrays of the same size. Each position is added, subtracted, multiplied, or divided with the matching position in the other array.

- Example: _1array = [10,20,30] and _2array = [50,60,70]. Result of _1array + _2array = [60, 80, 100].

```
import numpy as RNA
hehe = RNA.ones((4,4))
print(hehe)
print(".....................")
hehe[0] = 101
print(hehe)
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
.....................
[[101. 101. 101. 101.]
 [  1.   1.   1.   1.]
 [  1.   1.   1.   1.]
 [  1.   1.   1.   1.]]
```

Broadcast Array: if ones to change more than one index value at a time

29/09/2024

```
[5]: #Array arithmatic operation
import numpy as RNA
_1array = RNA.array([10, 20, 30])
_2array = RNA.array([50, 60, 70])
result = _1array + _2array
print(result)
```

```
[ 60  80 100]
```

Array arithmetic operation

→perform -, *, / by yourself

Loading Data from Files

NumPy can read data directly from text files and CSV files into arrays.

From Text File

The genfromtxt() function loads data from a text file. It can handle missing or null values by filtering, removing, or replacing them. It is perfect for data loading and cleaning.

- RNA.genfromtxt("rawdata.txt", delimiter=","): Reads comma-separated data from a text file.
- fileread.astype("int32"): Converts the loaded float values into integers.
- You can also read character data with dtype=str and encoding=None to print text from a file word by word.

The **genfromtxt()** function is used to load data in a program from a text file.
It takes multiple argument values to clean the data of the text file.
It also has the ability to deal with missing or null values through the processes of filtering, removing, and replacing.

The **genfromtxt()** function from the Numpy module is perfect for **data loading and cleaning.**

From text/notepad/wordpad file

```
[27]: import numpy as RNA
fileread = RNA.genfromtxt('rawdata.txt', delimiter=',')
fileread
```

```
[27]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12., 13.,
       14., 15., 16., 17., 18., 19., 20., 21., 22., 23., 23., 24., 25.,
       26., 27., 27., 29., 30.])
```

```
[31]: import numpy as RNA
      fileread = RNA.genfromtxt('rawdata.txt', delimiter=',')
      print(fileread)
      print("..................................................")
      fileread = fileread.astype('int32')
      print(fileread)
```

```
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17. 18.
 19. 20. 21. 22. 23. 23. 24. 25. 26. 27. 27. 29. 30.]
..................................................
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 23
 24 25 26 27 27 29 30]
```

Previous output type is float, hence convert into intger

```
import numpy as RNA
fileread = RNA.genfromtxt('rawchar.txt', dtype=str, encoding = None, delimiter=",")
for i in fileread:
    print(i,end=" ")
```

```
Bangladesh  officially the People's Republic of Bangladesh  is a country in South Asia. It is the eighth-most populous country in the world and among the most densely populated with a population of 170 million in an area of 148 460 square kilometres (57 320 sq mi). Bangladesh shares land borders with India to the north  west  and east  and Myanmar to the southeast. To the south  it has a coastline along the Bay of Bengal. It is separated from Bhutan and Nepal by the Siliguri Corridor  and from China by the mountainous Indian state of Sikkim. Dhaka  the capital and largest city  is the nation's political  financial  and cultural centre. Chittagong is the second-largest city and the busiest port. The official language is Bengali  with Bangladeshi English also used in government.
```

From CSV File

To load data from a CSV file, import the loadtxt library and open the file with open(), then pass it to loadtxt().

- from numpy import loadtxt
- csvread = open("csv1.csv", "rb")
- read = loadtxt(csvread, delimiter=",")

The loadtxt library must be imported separately. The open() function opens the CSV file, and loadtxt reads and parses it into a NumPy array.

From CSV

```
[78]: from numpy import loadtxt
      import numpy as RNA
      csvread = open('csv1.csv','rb')
      read = loadtxt(csvread, delimiter=',')
      print(read)
```

```
[[ 1. 10. 20. 30.]
 [ 2. 11. 21. 31.]
 [ 3. 12. 22. 32.]
 [ 4. 13. 23. 33.]
 [ 5. 14. 24. 34.]
 [ 6. 15. 25. 35.]
 [ 7. 16. 26. 36.]
 [ 8. 17. 27. 37.]
 [ 9. 18. 28. 38.]
 [10. 19. 29. 39.]]
```

loadtxt library file has to be imported if you want to load data from csv file. After that "csv" file will be accessed using open() function.

9. Pandas

What is Pandas?

Pandas is a Python library for working with data sets, especially tables (like a spreadsheet). It was created by Wes McKinney in 2008. The name comes from Panel Data and Python Data Analysis. You import it using: import pandas as pd (or any short name you choose).

What is Pandas?

→ Pandas is a Python library used for working with data sets

→ It has functions for analyzing, cleaning, exploring, and manipulating data

→ The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008

Why Use Pandas?

→ Pandas allows us to **analyze big data and make conclusions based on statistical theories**.

→ Pandas can **clean messy data sets, and make them readable** and relevant

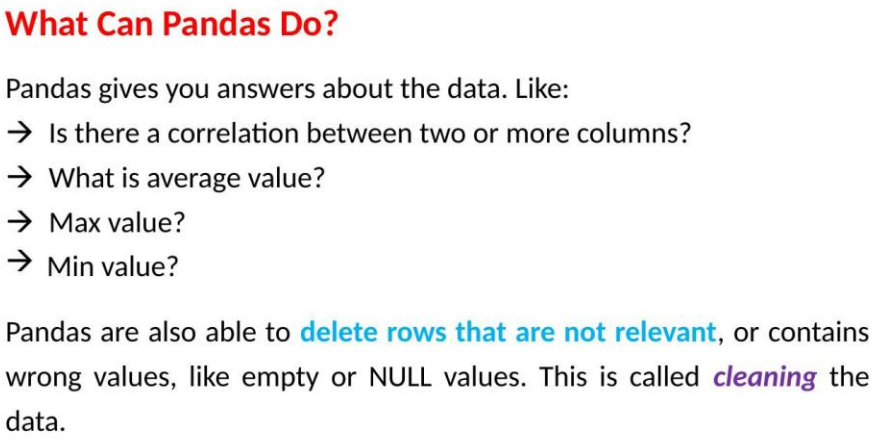
→ Relevant data is very **important in data science**

What Can Pandas Do?

Pandas can answer questions about your data such as:

- Is there a correlation between two or more columns?
- What is the average (mean) value of a column?
- What is the maximum or minimum value?

Pandas can also delete rows that are not relevant, or contain wrong or empty (NULL) values. This process is called cleaning the data.

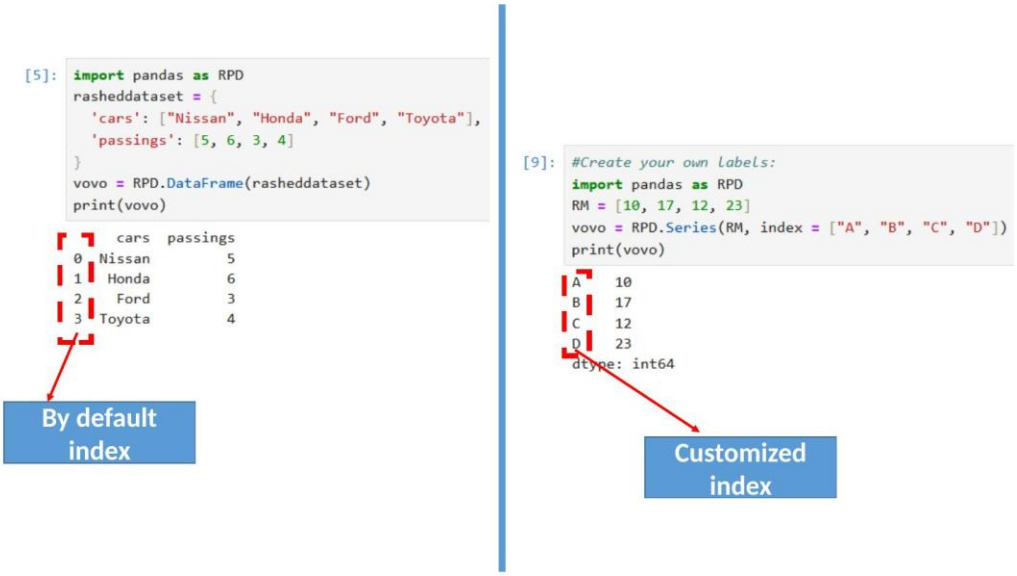


Series and DataFrame

The two main data structures in Pandas are Series and DataFrame.

- Series: A single column of data, like one list of values. You can give it custom labels (index). Example: RPD.Series([10,17,12,23], index=["A","B","C","D"])
- DataFrame: A full table with rows and columns. It is like a 2D spreadsheet. You create it from a dictionary where each key is a column name and the value is a list of data.

By default, rows are numbered starting from 0. You can also give custom row labels using the index parameter.



Selecting Specific Rows

Use .loc[] to select specific rows by their index number or label.

- vovo.loc[[0, 1]] prints rows at index 0 and 1.
- You can also give custom row names: RPD.DataFrame(data, index=["i","ii","iii"])

[15]:
```
#to print specific row
import pandas as RPD
data = {
  "Student_ID": [2024101, 2024102, 2024103],
  "Height": [5, 5.5, 6]
}
#load data into a DataFrame object:
vovo = RPD.DataFrame(data)
print(vovo.loc[[0, 1]])
```

```
   Student_ID  Height
0     2024101     5.0
1     2024102     5.5
```

[17]:
```
#index naming
import pandas as RPD
data = {
  "Student_ID": [2024101, 2024102, 2024103],
  "Height": [5, 5.5, 6]
}
vovo = RPD.DataFrame(data, index = ["i", "ii", "iii"])
print(vovo)
```

```
     Student_ID  Height
i       2024101     5.0
ii      2024102     5.5
iii     2024103     6.0
```

Reading CSV Files with Pandas

Pandas makes it very easy to read data from a CSV file into a DataFrame.

- RPD.read_csv("filename.csv"): Reads the entire CSV file into a table.
- vovo.head(5): Shows the first 5 rows of the table.
- vovo.tail(): Shows the last 5 rows of the table.
- RPD.options.display.max_rows: Shows how many rows Pandas will display at once (default is 60).

[25]:
```
# Data read from CSV using panda
import pandas as RPD
vovo = RPD.read_csv('pandacsv1.csv')
print(vovo)
```

```
         Name   ID Gender    Address
0      Rashed  101      M     Mirpur
1        Abir  102      M      Savar
2      Fatema  103      F     Banani
3       Robin  104      M     Paltan
4       Laila  105      F    Gulshan
5       Karim  106      M      Badda
6   Debashish  107      M     Kakoli
7        Naim  108      M  Dhanmondi
8      Farhad  109      M    Rampura
```

[27]:
```
# How many rows at a time (60)
print(RPD.options.display.max_rows)
```

```
60
```

[29]:
```
# view, for example first 5 rows
import pandas as RPD
vovo = RPD.read_csv('pandacsv1.csv')
print(vovo.head(5))
```

```
     Name   ID Gender  Address
0  Rashed  101      M   Mirpur
1    Abir  102      M    Savar
2  Fatema  103      F   Banani
3   Robin  104      M   Paltan
4   Laila  105      F  Gulshan
```

[31]:
```
# view last 5 rows
print(vovo.tail())
```

```
         Name   ID Gender    Address
4       Laila  105      F    Gulshan
5       Karim  106      M      Badda
6   Debashish  107      M     Kakoli
7        Naim  108      M  Dhanmondi
8      Farhad  109      M    Rampura
```

Cleaning Data: Handling Empty Cells

Empty cells in a dataset cause problems in analysis. Pandas gives you several ways to deal with them.

Method 1: dropna()

This removes all rows that have any empty (NaN) cell. It returns a new DataFrame and does not change the original.

- hehe = vovo.dropna()

```
[35]: # Return a new Data Frame with no empty cells
      # By default, the dropna() method returns a new DataFrame, and will not change the original.
      import pandas as RPD
      vovo = RPD.read_csv('panda_practice.csv')
      hehe = vovo.dropna()
      print(hehe.head(20))
```

```
    Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
5         60    102       127     300.0
6         60    110       136     374.0
7         45    104       134     253.3
8         30    109       133     195.1
9         60     98       124     269.0
10        60    103       147     329.3
11        60    100       120     250.7
12        60    106       128     345.3
13        60    104       132     379.3
14        60     98       123     275.0
15        60     98       120     215.2
16        60    100       120     300.0
18        60    103       123     323.0
19        45     97       125     243.0
20        60    108       131     364.2
```

Method 2: fillna()

Instead of removing rows, you can fill empty cells with a value you choose.

- vovo.fillna(130, inplace=True): Fills all empty cells in the whole table with the value 130.
- vovo.fillna({"Calories": 150}, inplace=True): Fills empty cells only in the Calories column with 150.
- vovo.fillna({"Calories": vovo["Calories"].mean()}, inplace=True): Fills empty cells with the average (mean) value of that column. This is a smarter and more common approach.

You can also fill with median or mode. The median is the middle value. The mode is the most repeated value. Use .median() or .mode()[0] in the same way.

02/10/2024

```
[2]: import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
vovo.fillna(130, inplace = True)
print(vovo.head(20))
```

```
    Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
5         60    102       127     300.0
6         60    110       136     374.0
7         45    104       134     253.3
8         30    109       133     195.1
9         60     98       124     269.0
10        60    103       147     329.3
11        60    100       120     250.7
12        60    106       128     345.3
13        60    104       132     379.3
14        60     98       123     275.0
15        60     98       120     215.2
16        60    100       120     300.0
17        45     90       112     130.0
18        60    103       123     323.0
19        45     97       125     243.0
```

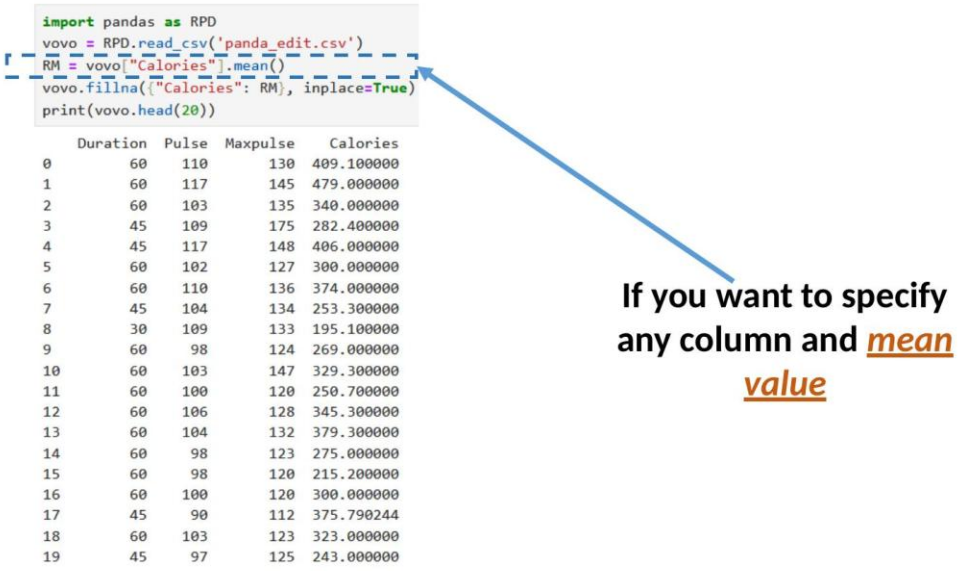
Another way of dealing with empty cells is to insert a new value instead.

The fillna() method allows us to replace all empty cells with a value

```
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
vovo.fillna({"Calories": 150}, inplace=True)
print(vovo.head(20))
```

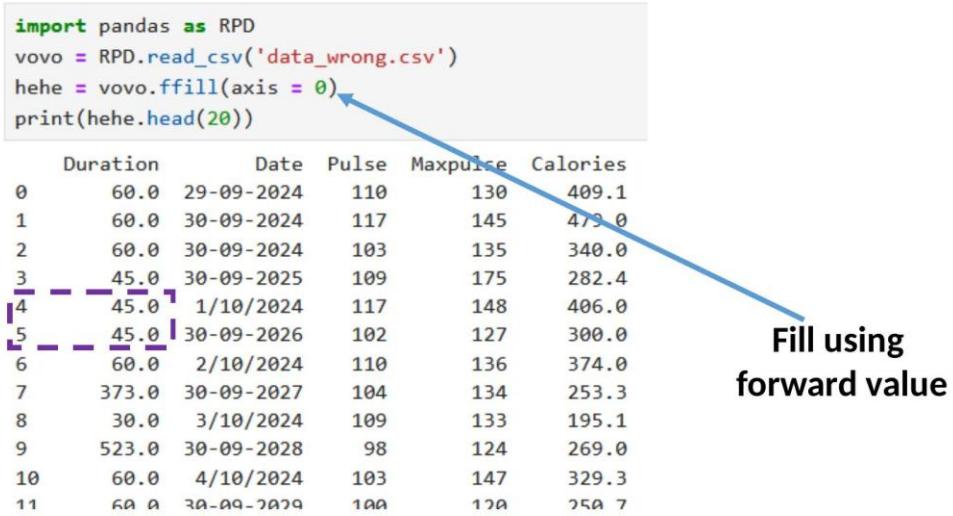
```
    Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
5         60    102       127     300.0
6         60    110       136     374.0
7         45    104       134     253.3
8         30    109       133     195.1
9         60     98       124     269.0
10        60    103       147     329.3
11        60    100       120     250.7
12        60    106       128     345.3
13        60    104       132     379.3
14        60     98       123     275.0
15        60     98       120     215.2
16        60    100       120     300.0
17        45     90       112     150.0
18        60    103       123     323.0
19        45     97       125     243.0
```

If you want to specify any column



Method 3: Forward Fill and Backward Fill

- vovo.ffill(axis=0): Forward fill. Fills each empty cell with the value from the row above it.
- Backward fill (bfill): Fills each empty cell with the value from the row below it.



Cleaning Data: Fixing Wrong Values

Sometimes data has wrong values, not missing ones. For example, a Duration value of 373 or 523 that should be 60. You can fix these directly using .loc[].

- vovo.loc[7:9, "Duration"] = 29: Sets rows 7, 8, and 9 in the Duration column to 29.

For larger datasets, use a loop with a condition to find and fix wrong values automatically.

- If Duration is greater than 59, replace it with 22.

You can also drop rows where the value is wrong instead of fixing them.

- vovo.drop(rm, inplace=True): Removes a row that meets a condition.

```
#Wrong data --> Type mistake
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
print(vovo.head(20))
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	29-09-2024	110	130	409.1
1	60	30-09-2024	117	145	479.0
2	60	30-09-2024	103	135	340.0
3	45	30-09-2025	109	175	282.4
4	45	1/10/2024	117	148	406.0
5	60	30-09-2026	102	127	300.0
6	60	2/10/2024	110	136	374.0
7	29	30-09-2027	104	134	253.3
8	29	3/10/2024	109	133	195.1
9	29	30-09-2028	98	124	269.0
10	60	4/10/2024	103	147	329.3
11	60	30-09-2029	100	120	250.7
12	60	5/10/2024	106	128	345.3
13	60	30-09-2030	104	132	379.3

Try to find out wrong value and make correction

```
#Wrong data --> Type mistake ---> if the data set is large enough --> need to set condition
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
for rm in vovo.index:
  if vovo.loc[rm, "Duration"] > 59:
    vovo.loc[rm, "Duration"] = 22
print(vovo.head(20))
```

	Duration	Date	Pulse	Maxpulse	Calories
0	22	29-09-2024	110	130	409.1
1	22	30-09-2024	117	145	479.0
2	22	30-09-2024	103	135	340.0
3	45	30-09-2025	109	175	282.4
4	45	1/10/2024	117	148	406.0
5	22	30-09-2026	102	127	300.0
6	22	2/10/2024	110	136	374.0
7	29	30-09-2027	104	134	253.3
8	29	3/10/2024	109	133	195.1
9	29	30-09-2028	98	124	269.0
10	22	4/10/2024	103	147	329.3
11	22	30-09-2029	100	120	250.7
12	22	5/10/2024	106	128	345.3
13	22	30-09-2030	104	132	379.3
14	22	6/10/2024	98	123	275.0
15	22	30-09-2031	98	120	215.2
16	22	7/10/2024	100	120	300.0
17	45	30-09-2032	90	112	NaN
18	22	8/10/2024	103	123	323.0
19	45	30-09-2033	97	125	243.0

Try to find out wrong value and make correction

***** based on certain condition***

```python
import pandas as RPD
vovo = RPD.read_csv('data_wrong.csv')
vovo.loc[7:9, 'Duration'] = 29
for rm in vovo.index:
  if vovo.loc[rm, "Duration"] > 59:
    vovo.drop(rm, inplace = True)
print(vovo)
```

```
     Duration        Date  Pulse  Maxpulse  Calories
3          45  30-09-2025    109       175     282.4
4          45   1/10/2024    117       148     406.0
7          29  30-09-2027    104       134     253.3
8          29   3/10/2024    109       133     195.1
9          29  30-09-2028     98       124     269.0
..        ...         ...    ...       ...       ...
159        30  30-09-2103     80       120     240.9
160        30  18-12-2024     85       120     250.4
161        45  30-09-2104     90       130     260.4
162        45  19-12-2024     95       130     270.0
163        45  30-09-2105    100       140     280.9

[64 rows x 5 columns]
```

Drop the rows
Based on
Certain condition

Handling Duplicate Rows

Sometimes the same row appears more than once in a dataset. Pandas helps you find and remove these duplicate rows.

- vovo.duplicated(): Returns True for every row that is a duplicate, and False for rows that are unique.
- bobo.drop_duplicates(inplace=True): Removes all duplicate rows from the table permanently.

Handling of Duplicates value

→ The duplicated() method returns a Boolean values for each row

For example,

- Returns **True for every row** that is a duplicate, **otherwise False**

```
# handling of duplicate value (current data set view)
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
print(vovo.head(40))
```

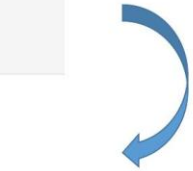
```
5     60  102  127  300.0
6     60  110  136  374.0
7     45  104  134  253.3
8     30  109  133  195.1
9     60   98  124  269.0
10    60  103  147  329.3
11    60  100  120  250.7
12    60  106  128  345.3
13    60  104  132  379.3
14    60   98  123  275.0
15    60   98  120  215.2
16    60  100  120  300.0
17    45   90  112    NaN
18    60  103  123  323.0
19    45   97  125  243.0
20    60  108  131  364.2
21    45  100  119  282.0
22    60  130  101  300.0
23    45  105  132  246.0
24    60  102  126  334.5
25    60  100  120  250.0
26    60   92  118  241.0
27    60  103  132    NaN
28    60  100  132  280.0
29    60  102  129  380.3
30    60   92  115  243.0
31    45   90  112  180.1
32    60  101  124  299.0
33    60   93  113  223.0
34    60  107  136  361.0
35    60  114  140  415.0
36    60  102  127  300.0
37    60  100  120  300.0
38    60  100  120  300.0
39    45  104  129  266.0
```

```
[8]: # handling of duplicate value (boolean value, true for duplicate)
import pandas as RPD
vovo = RPD.read_csv('panda_edit.csv')
popo = vovo.duplicated()
print(popo.head(40))
```

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
16    False
```

```
# handling of duplicate value (delete duplicate values)
import pandas as RPD
bobo = RPD.read_csv('panda_edit.csv')
bobo.drop_duplicates(inplace = True)
print(bobo.head(40))
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0



28	60	100	132	280.0
29	60	102	129	380.3
30	60	92	115	243.0
31	45	90	112	180.1
32	60	101	124	299.0
33	60	93	113	223.0
34	60	107	136	361.0
35	60	114	140	415.0
39	45	104	129	266.0
41	60	98	126	286.0
42	60	100	122	329.4
43	60	111	138	400.0

Plotting with Pandas

Pandas has a built-in plot() method to create charts directly from your data. It works together with the matplotlib library to display the chart on screen.

Line Chart

vovo.plot() draws a line chart for all columns in the DataFrame. Each column becomes one colored line on the chart.

Plotting:
Pandas uses the plot() method to create diagrams.

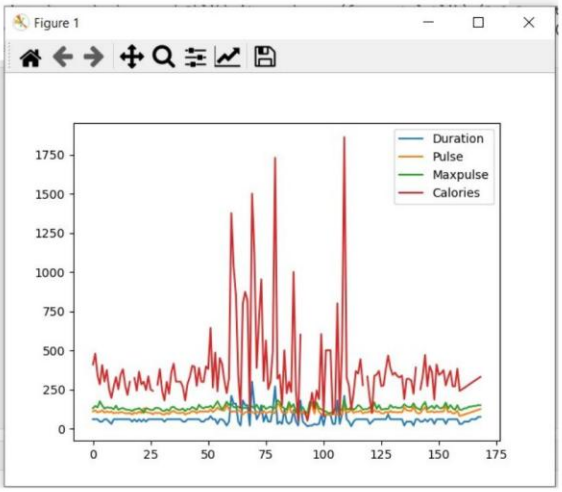
```
# pandas: graphical representation
# type, pip install matplotlib
# type, conda install -c anaconda pyqt
# type, pip install PyQt6
import sys
import matplotlib
matplotlib.use('Qt5Agg')

import pandas as RPD
import matplotlib.pyplot as PLOT

vovo = RPD.read_csv('panda_edit.csv')

vovo.plot()

PLOT.show()
```



Scatter Plot

A scatter plot shows the relationship between two specific columns as dots on a graph.

- vovo.plot(kind="scatter", x="Duration", y="Calories"): Plots Duration on the x-axis and Calories on the y-axis.

Plotting:
Pandas uses the plot() scatter graph-> two columns based

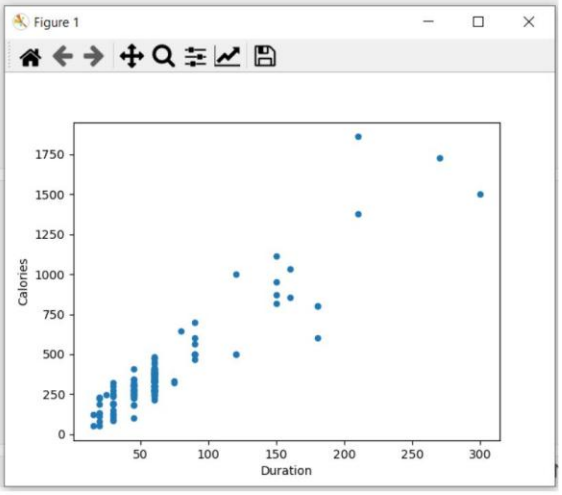
```
# for particular columns --> scatter graph
import sys
import matplotlib
matplotlib.use('Qt5Agg')

import pandas as RPD
import matplotlib.pyplot as PLT

vovo = RPD.read_csv('panda_edit.csv')

vovo.plot(kind = 'scatter', x = 'Duration', y = 'Calories')

PLT.show()
```



Histogram

A histogram shows how frequently different value ranges appear in one column.

- vovo["Calories"].plot(kind="hist"): Creates a histogram for the Calories column only.

Plotting:

Pandas uses the plot()

histogram:

based on single column

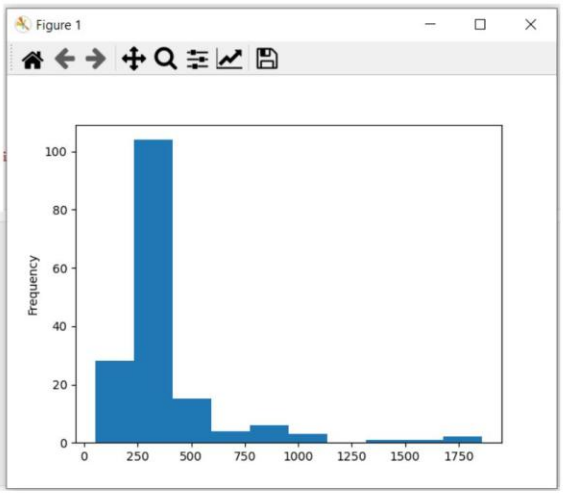
```
#histogram based on single column
import sys
import matplotlib
matplotlib.use('Qt5Agg')

import pandas as RPD
import matplotlib.pyplot as PLT

vovo = RPD.read_csv('panda_edit.csv')

vovo["Calories"].plot(kind = 'hist')

PLT.show()
```



End of Notes